BIG DATA ANALYTICS

Tuần 2 — Phần 1

Nền tảng Big Data & Data Mining

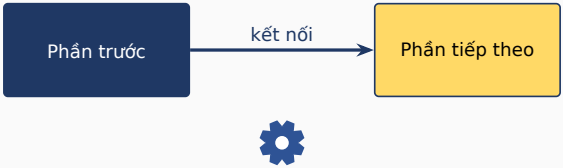
Khái niệm cốt lõi, giới hạn thống kê khi khai phá dữ liệu lớn,
và bốn kỹ thuật khai phá dữ liệu nền tảng



Biên soạn tham khảo trên nền đề cương BDAN437764 — Big Data Analytics

Tài liệu tham khảo chính: Han–Kamber–Pei (2011); Leskovec–Rajaraman–Ullman, *Mining of Massive Datasets* (Chương 1, mmds.org); Stanford CS246

Kết nối từ Tuần 1



Tuần 1 đã cho chúng ta bức tranh toàn cảnh (roadmap 7 phần) và môi trường thực hành sẵn sàng. Hôm nay chúng ta bắt đầu đi sâu vào phần đầu tiên:

- Big Data thực chất là gì, và vì sao nó khác với "nhiều dữ liệu" thông thường?
- Data mining — kỹ thuật rút tri thức từ dữ liệu — hoạt động theo nguyên lý nào, và giới hạn của nó ở đâu?
- Đây là nền tảng khái niệm cho toàn bộ 6 phần kỹ thuật còn lại của môn học

Nội dung buổi học Tuần 2 — Phần 1

- Lý thuyết (3 tiết)
  - Data mining là gì — bốn cách tiếp cận, câu chuyện John Snow
  - Đặc trưng Big Data — mô hình 5V và vì sao RDBMS không còn đủ
  - Giới hạn thống kê khi khai phá dữ liệu lớn — nguyên lý Bonferroni
  - Kiến thức nền tảng cần biết: TF.IDF, hash function, index, secondary storage, power law
  - Toàn cảnh hệ sinh thái công nghệ Big Data hiện đại
  - Bốn kỹ thuật khai phá dữ liệu nền tảng: classification, clustering, association rules, regression
- Thực hành (2 tiết)
  - Case study: phân tích 5V trên một hệ thống dữ liệu lớn thực tế
  - Bài tập nhận diện & phân loại bài toán khai phá dữ liệu
  - 6 mini-lab bằng Python: TF.IDF, mô phỏng Bonferroni, clustering, association rules, classification, regression

Mục tiêu học tập (CLO) của Phần 1

- Trình bày và phân loại được khái niệm, đặc trưng của Big Data (mô hình 5V)
- Giải thích được nguyên lý Bonferroni và vì sao phải thận trọng khi "tìm mẫu" trong dữ liệu quy mô lớn
- Vận dụng được các công cụ nền tảng (TF.IDF, hash function, index) trong bối cảnh khai phá dữ liệu
- Phân biệt được bốn kỹ thuật khai phá dữ liệu nền tảng và nhận diện kỹ thuật phù hợp cho một bài toán cụ thể
- Phân loại được một bài toán Big Data theo đặc điểm dữ liệu và mô hình tính toán phù hợp

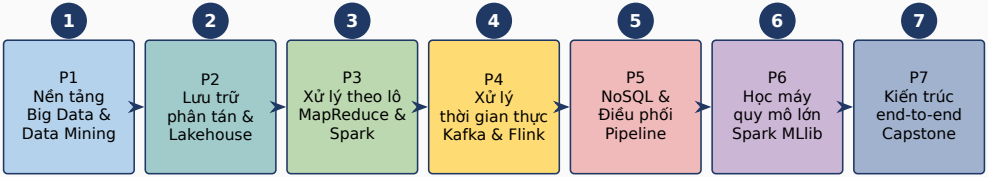
Vị trí của Phần 1 trong Roadmap

- Phần 1 là **nền tảng khái niệm** cho toàn bộ 6 phần còn lại của môn học
- Mọi công nghệ sắp học (HDFS, Spark, Kafka...) đều là công cụ để giải quyết các vấn đề Volume/Velocity/Variety sẽ trình bày hôm nay
- Các kỹ thuật data mining học hôm nay sẽ được "phóng to" lên quy mô phân tán ở Phần 3 và Phần 6



DATA MINING LÀ GÌ?

Bốn cách tiếp cận để biến dữ liệu thô thành tri thức



Định nghĩa Data Mining

- Định nghĩa được chấp nhận rộng rãi nhất: data mining là **sự khám phá các "mô hình" (model) cho dữ liệu**
- "Mô hình" ở đây có thể là nhiều thứ khác nhau — không có một định nghĩa toán học duy nhất
- Nguồn: Leskovec, Rajaraman, Ullman — *Mining of Massive Datasets*, Chương 1, Mục 1.1 (mmds.org)

Data Mining — một khái niệm, nhiều tên gọi

Data Mining

$\approx$ Predictive Analytics

$\approx$ Data Science

$\approx$ Machine Learning

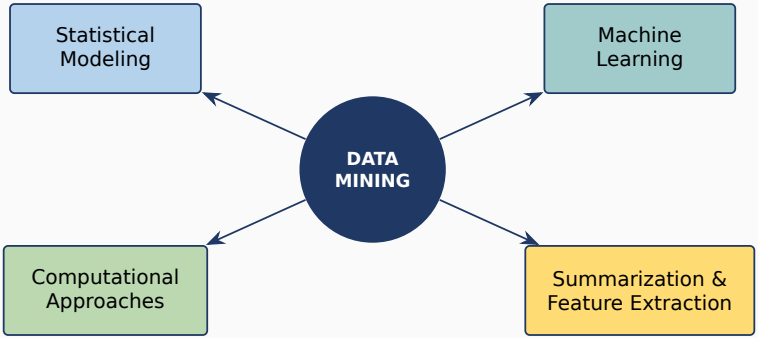
$\approx$ Data-Centric AI

Nguồn: Stanford CS246 — Mining Massive Datasets (Jure Leskovec), bài giảng 1

Bốn cách tiếp cận để xây dựng "mô hình"

- Các ngành khoa học khác nhau nhìn nhận data mining theo những cách khác nhau
- Sách *Mining of Massive Datasets* tổng hợp thành bốn hướng chính:
- 1. Statistical Modeling 2. Machine Learning
- 3. Computational Approaches to Modeling
- 4. Summarization & Feature Extraction
- Bốn hướng này không loại trừ nhau — một hệ thống thực tế thường kết hợp nhiều hướng

Bốn cách tiếp cận mô hình hóa dữ liệu



Cả bốn hướng đều nhằm trả lời một câu hỏi chung: đâu là cấu trúc ẩn phía sau dữ liệu quan sát được?

1. Statistical Modeling

- Thống kê học là ngành đầu tiên sử dụng thuật ngữ "data mining" — ban đầu mang nghĩa **tiêu cực** ("data dredging": cố tình khai thác dữ liệu để tìm ra điều không thực sự tồn tại)
- Ngày nay: các nhà thống kê xem data mining là việc xây dựng một **mô hình thống kê** — tức một phân bố xác suất giả định mà dữ liệu quan sát được xem là được sinh ra từ đó
- *Ví dụ*: giả sử dữ liệu là một tập số — ta có thể giả định chúng tuân theo phân bố Gauss, và tính tham số (trung bình, độ lệch chuẩn) đặc trưng cho phân bố đó

- Xem dữ liệu như **tập huấn luyện** (training set) cho một thuật toán học máy (Bayes net, SVM, decision tree, hidden Markov model,...)
- **Hiệu quả khi**: ta không có ý tưởng rõ ràng về "cái gì" cần tìm — ví dụ bài toán Netflix Challenge (dự đoán đánh giá phim của người dùng) không có công thức tường minh, nhưng học máy giải quyết rất tốt
- **Không hiệu quả khi** mục tiêu có thể mô tả trực tiếp: startup WhizBang! Labs từng dùng học máy để nhận diện resume trên Web, nhưng không vượt qua được thuật toán thiết kế thủ công tìm từ khóa đặc trưng

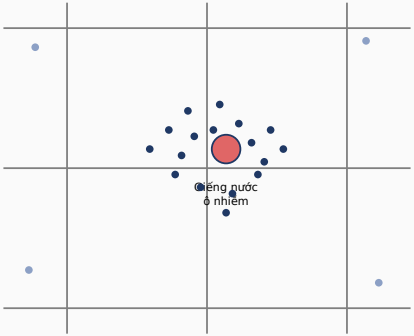
3. Computational Approaches to Modeling

- Các nhà khoa học máy tính xem data mining là một **bài toán thuật toán**: mô hình chính là câu trả lời cho một truy vấn phức tạp về dữ liệu
- *Ví dụ*: với tập số ở trên, thay vì giả định phân bố Gauss, ta có thể chỉ đơn giản **tính** trung bình và độ lệch chuẩn — những con số này có thể không phải tham số tối ưu của phân bố Gauss, nhưng sẽ rất gần nếu dữ liệu đủ lớn
- Cách tiếp cận này gắn liền với việc thiết kế thuật toán **hiệu quả** trên dữ liệu lớn — chủ đề xuyên suốt Phần 3 (MapReduce & Spark)

4a. Summarization — tóm tắt dữ liệu

- Một trong những dạng tóm tắt thú vị nhất: **PageRank** — toàn bộ cấu trúc phức tạp của Web được tóm tắt bằng một con số duy nhất cho mỗi trang (xác suất một người lướt web ngẫu nhiên đang ở trang đó)
- Một dạng tóm tắt quan trọng khác: **Clustering** — dữ liệu được xem như các điểm trong không gian nhiều chiều; các điểm "gần" nhau được gom vào cùng một cụm, và cụm được tóm tắt bằng tâm cụm (centroid)

Case study: John Snow và bản đồ dịch tả London (1854)



Các ca bệnh (chấm xanh) tập trung quanh một giếng nước — đây là ví dụ kinh điển về "clustering" được thực hiện hoàn toàn thủ công, không cần máy tính

Case study: John Snow — vì sao đây là data mining?

- Bác sĩ John Snow đối mặt với một đợt dịch tả bùng phát tại London, không rõ nguyên nhân
- Ông vẽ từng ca bệnh lên bản đồ thành phố — một hình thức **trực quan hóa** và **tóm tắt** dữ liệu
- Kết quả: các ca bệnh **tập trung** quanh một số giao lộ nhất định — chính là vị trí của các giếng nước bị ô nhiễm
- Nếu không có khả năng "gom cụm" dữ liệu theo không gian, nguyên nhân dịch tả có thể đã không được phát hiện
- Bài học: **summarization/clustering** là công cụ mạnh để tìm nguyên nhân ẩn trong dữ liệu quan sát — nguyên lý này không đổi dù dữ liệu là hàng chục ca bệnh hay hàng tỷ bản ghi

4b. Feature Extraction — trích xuất đặc trưng nổi bật

- Frequent Itemsets (tập mục thường xuyên)
  - Phù hợp khi dữ liệu là các "giỏ hàng" (basket) chứa nhiều mục nhỏ
  - Tìm các tập mục nhỏ xuất hiện cùng nhau thường xuyên — nền tảng của **association rules** sẽ học ở phần sau
- Similar Items (các mục tương tự)
  - Dữ liệu được xem như tập hợp, mục tiêu là tìm các cặp có nhiều điểm chung
  - *Ví dụ*: Amazon tìm khách hàng "tương tự" nhau về sở thích để gợi ý sản phẩm — kỹ thuật này gọi là **collaborative filtering**

Descriptive vs. Predictive Methods

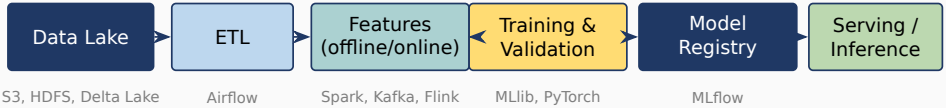
Descriptive (Mô tả)

- Tìm các mẫu (pattern) **con người có thể diễn giải được** để mô tả dữ liệu
- Không nhất thiết dự đoán điều gì mới
- Ví dụ điển hình: **Clustering**

Predictive (Dự đoán)

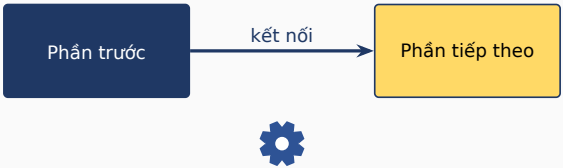
- Dùng một số biến để dự đoán giá trị **chưa biết hoặc tương lai** của biến khác
- Cần một tập huấn luyện có nhãn hoặc kết quả đã biết
- Ví dụ điển hình: **Recommender Systems**

Data Mining trong pipeline ML hiện đại



Data mining không đứng riêng lẻ — nó là một mắt xích trong pipeline lớn hơn: từ Data Lake, qua ETL/Feature Engineering, đến Training và Serving. Mỗi khối sẽ có một phần riêng trong roadmap (P2, P4, P5, P6)

Cầu nối sang phần tiếp theo

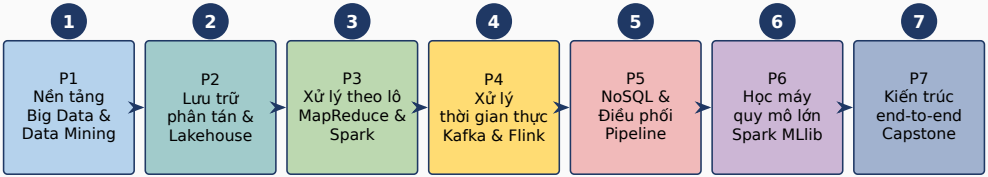


*Đã hiểu **data mining là gì** và **bốn cách tiếp cận** để xây dựng mô hình. Nhưng câu hỏi tiếp theo là:*

- Dữ liệu phải "lớn" đến mức nào thì mới thực sự là **Big Data**?
- Big Data khác gì với "nhiều dữ liệu" một cách đơn thuần?
- $\rightarrow$ Câu trả lời nằm ở mô hình **5V**

ĐẶC TRƯNG CỦA BIG DATA — MÔ HÌNH 5V

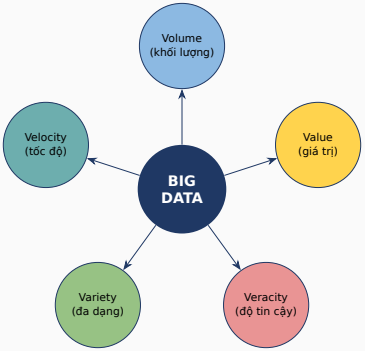
Khung tư duy để đánh giá một bài toán có thực sự "đủ Big" hay không



Định nghĩa hoạt động của Big Data

- Big Data: tập dữ liệu có quy mô, tốc độ phát sinh, hoặc mức độ đa dạng **vượt quá khả năng xử lý** của công cụ quản trị dữ liệu truyền thống (một máy chủ, một CSDL quan hệ)
- Đây không phải một ngưỡng cố định về số GB/TB — mà là một **ngưỡng tương đối**, phụ thuộc vào hạ tầng và bài toán cụ thể
- Khái niệm "3V" gốc (Volume, Velocity, Variety) do Doug Laney (hãng nghiên cứu Gartner) đề xuất năm 2001
- Về sau mở rộng thành 5V khi ngành công nghiệp nhận ra chất lượng (Veracity) và mục đích sử dụng dữ liệu (Value) cũng quan trọng không kém quy mô

Năm đặc trưng của Big Data (5V)



5V không phải là định nghĩa cứng nhắc, mà là khung tư duy để đánh giá một bài toán

Volume (khối lượng)

- Quy mô dữ liệu vượt khả năng lưu trữ/xử lý của một máy đơn lẻ
- Đo bằng terabyte (TB), petabyte (PB), thậm chí exabyte (EB)
- *Ví dụ*: một sàn thương mại điện tử lớn có thể tích lũy hàng chục TB log hành vi người dùng mỗi ngày
- Câu hỏi kiểm tra: dữ liệu này có còn **vừa trong RAM** của một máy chủ mạnh hay không?

Velocity (tốc độ)

- Tốc độ dữ liệu phát sinh **và** tốc độ cần được xử lý/phản hồi (latency)
- Hai khía cạnh: tốc độ nạp dữ liệu vào hệ thống (ingestion rate) và độ trễ cho phép khi trả lời truy vấn
- *Ví dụ*: một giao dịch thẻ ngân hàng cần được đánh giá gian lận trong **vài trăm mili-giây** — không thể chờ xử lý theo lô (batch) vào cuối ngày
- Đây chính là động lực ra đời của xử lý thời gian thực — chủ đề của Phần 4

Variety (đa dạng)

- Ba mức độ cấu trúc dữ liệu:
  - Có cấu trúc (structured) — bảng quan hệ, schema cố định
  - Bán cấu trúc (semi-structured) — JSON, XML, log file
  - Phi cấu trúc (unstructured) — văn bản tự do, hình ảnh, âm thanh, video
- Dữ liệu đến từ nhiều nguồn khác nhau: cảm biến IoT, mạng xã hội, giao dịch, hệ thống nội bộ, log ứng dụng
- Một hệ thống RDBMS truyền thống chỉ xử lý tốt dữ liệu có cấu trúc — đây là động lực cho Data Lake (Phần 2)

Veracity (độ tin cậy)

- Chất lượng và độ chính xác của dữ liệu — dữ liệu có thể bị nhiễu, thiếu, trùng lặp, hoặc sai lệch
- *Ví dụ*: cảm biến IoT có thể lỗi phần cứng và gửi giá trị vô lý; dữ liệu mạng xã hội có thể chứa tin giả hoặc tài khoản ảo
- Nguyên tắc kinh điển: **"garbage in, garbage out"** — mô hình khai phá dữ liệu không thể tốt hơn chất lượng dữ liệu đầu vào
- Veracity liên hệ trực tiếp đến phần **Giới hạn thống kê** sắp học — dữ liệu nhiễu càng dễ tạo ra "mẫu giả"

Value — chữ V quan trọng nhất

- Bốn chữ V đầu mô tả **đặc điểm** của dữ liệu; Value mô tả **mục đích** thu thập và xử lý nó
- Dữ liệu chỉ có giá trị khi được chuyển hóa thành tri thức hoặc quyết định hữu ích — đây chính là vai trò của **Big Data Analytics**, mục tiêu xuyên suốt học phần này
- Toàn bộ 7 phần trong roadmap là các bước kỹ thuật để đi từ dữ liệu thô (Volume/Velocity/Variety) đến giá trị (Value), với Veracity là điều kiện ràng buộc xuyên suốt

Tổng hợp 5V — câu hỏi tự kiểm tra

Đặc trưng	Câu hỏi kiểm tra nhanh
Volume	Dữ liệu có còn vừa trong RAM/đĩa một máy không?
Velocity	Có cần phản hồi trong thời gian thực không?
Variety	Dữ liệu có nhiều định dạng khác nhau không?
Veracity	Dữ liệu có đáng tin cậy, đã được làm sạch chưa?
Value	Kết quả phân tích có tạo ra quyết định/giá trị cụ thể không?

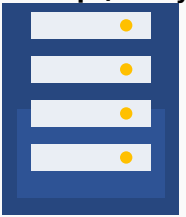
Áp dụng 5V cho một hệ thống thực tế: mạng xã hội

- **Volume**: hàng tỷ bài đăng, bình luận, lượt tương tác mỗi ngày
- **Velocity**: newsfeed cần cập nhật gần như tức thời
- **Variety**: văn bản, hình ảnh, video, dữ liệu đồ thị quan hệ bạn bè — rất cao
- **Veracity**: tin giả, tài khoản ảo là thách thức lớn
- **Value**: xếp hạng nội dung phù hợp, phát hiện thông tin độc hại



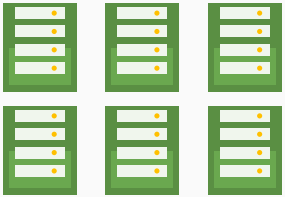
Vì sao RDBMS không còn đủ?

Scale-up (1 máy)



CPU/RAM/Disk
có giới hạn vật lý

Scale-out (nhiều máy)



Mở rộng gần như
không giới hạn

RDBMS truyền thống thiết kế để scale-up (nâng cấp một máy); Big Data cần scale-out (phân tán trên nhiều máy)

Scale-up vs. Scale-out

Scale-up (RDBMS truyền thống)

- Nâng cấp CPU/RAM/đĩa của **một máy duy nhất**
- Đơn giản về vận hành, nhất quán dữ liệu mạnh (ACID)
- Có **trần vật lý** — không thể mở rộng vô hạn

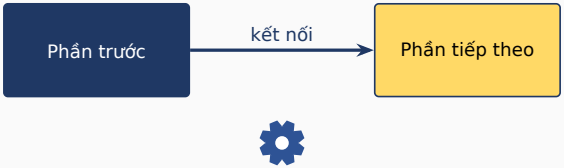
Scale-out (hệ sinh thái Big Data)

- Phân tán dữ liệu & tính toán trên **nhiều máy rẻ tiền**
- Mở rộng gần như không giới hạn bằng cách thêm máy
- Đánh đổi: phức tạp hơn về vận hành, nhất quán, và chịu lỗi

Đánh đổi khi chuyển sang hệ phân tán

- Không có "bữa trưa miễn phí": đổi lấy khả năng mở rộng, ta phải chấp nhận độ phức tạp cao hơn
- Vấn đề chịu lỗi (fault tolerance): máy trong cluster có thể hỏng bất cứ lúc nào — hệ thống phải tự phục hồi (sẽ học ở Phần 2 với HDFS)
- Vấn đề nhất quán dữ liệu: khi dữ liệu được sao chép trên nhiều máy, đảm bảo tính nhất quán trở nên khó khăn hơn nhiều (định lý CAP — sẽ học ở Phần 5)
- Đây chính là lý do cả một hệ sinh thái công nghệ (Phần 2–7) ra đời để giải quyết những đánh đổi này một cách có hệ thống

Cầu nối sang phần tiếp theo

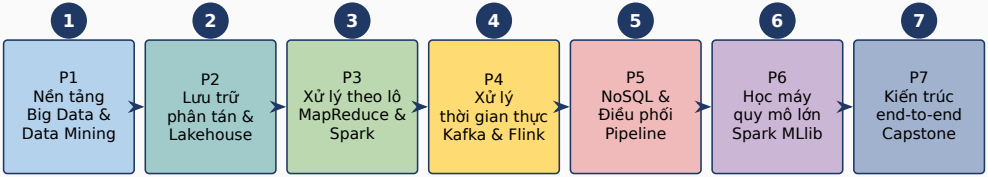


*5V mô tả **đặc điểm của dữ liệu**. Nhưng khi dữ liệu càng lớn, một nguy cơ khác xuất hiện:*

- Càng tìm kiếm nhiều "mẫu" trong một tập dữ liệu khổng lồ, càng dễ tìm thấy những mẫu **trông có vẻ ý nghĩa nhưng thực chất chỉ là ngẫu nhiên**
- $\rightarrow$ Đây là nội dung của **nguyên lý Bonferroni** — một trong những bài học quan trọng nhất khi làm việc với Big Data

GIỚI HẠN THỐNG KÊ KHI KHAI PHÁ DỮ LIỆU LỚN

Nguyên lý Bonferroni — vì sao "tìm mẫu" trong dữ liệu khổng lồ rất dễ gây hiểu lầm



Vấn đề: dữ liệu càng lớn, mẫu giả càng nhiều

- Một dạng bài toán khai phá dữ liệu phổ biến: **tìm sự kiện bất thường** ẩn trong khối dữ liệu khổng lồ
- Trực giác sai lầm thường gặp: "dữ liệu càng nhiều, càng dễ tìm ra sự thật"
- Thực tế: nếu tìm kiếm đủ rộng trong một tập dữ liệu đủ lớn, ta **gần như chắc chắn** sẽ tìm thấy những "mẫu" trông có vẻ ý nghĩa — ngay cả khi dữ liệu hoàn toàn ngẫu nhiên
- Nguồn: *Mining of Massive Datasets*, Mục 1.2 (mmds.org)

Câu chuyện thực tế: Total Information Awareness (TIA)

- Sau vụ khủng bố 11/9/2001, người ta phát hiện 4 kẻ khủng bố đã theo học tại các trường dạy lái máy bay thương mại, dù không thuộc hãng hàng không nào
- Giả thuyết: nếu khai thác đủ dữ liệu (hóa đơn thẻ tín dụng, hồ sơ khách sạn, dữ liệu di chuyển...), có thể **phát hiện sớm** hành vi khủng bố
- Chương trình **TIA (Total Information Awareness)** ra đời với mục tiêu này, nhưng gây tranh cãi lớn về quyền riêng tư và cuối cùng bị Quốc hội Mỹ hủy bỏ
- Câu hỏi kỹ thuật quan trọng: nếu tìm trong lượng dữ liệu khổng lồ những hành vi "trông giống" khủng bố, liệu ta có tìm ra nhiều hoạt động vô tội bị gắn nhãn sai không?

Nguyên lý Bonferroni — phát biểu trực quan

- Xuất phát từ một định lý thống kê chặt chẽ (Bonferroni correction), nhưng ta chỉ cần phiên bản trực quan:
- **Bước 1**: Tính *số lượng kỳ vọng* các sự kiện "trông giống" điều đang tìm, với giả định dữ liệu hoàn toàn **ngẫu nhiên**
- **Bước 2**: Nếu con số này **lớn hơn đáng kể** so với số thực thể thật sự ta hy vọng tìm được $\rightarrow$ hầu hết những gì tìm thấy sẽ là **giả** (statistical artifact), không phải bằng chứng thật
- Đây là một dạng cảnh báo: **không phải cứ tìm kiếm rộng trong dữ liệu lớn là kết quả đáng tin cậy**

Ví dụ minh họa: "kẻ xấu tại khách sạn" — thiết lập bài toán

- Giả sử có những "kẻ xấu" định kỳ gặp nhau tại khách sạn để bàn mưu — ta muốn tìm ra họ qua dữ liệu lưu trú
- Bốn giả định về quy mô dữ liệu:
  - Có 1 tỷ (10^9) người có thể là nghi phạm
  - Mỗi người đi khách sạn với xác suất $1/100$ mỗi ngày
  - Mỗi khách sạn chứa 100 người $\Rightarrow$ có 100.000 (10^5) khách sạn
  - Ta xem xét dữ liệu lưu trú trong 1.000 ngày
- Ta tìm những **cặp người** cùng ở một khách sạn vào **hai ngày khác nhau** — giả định **không có** kẻ xấu thật nào, mọi người hành xử hoàn toàn ngẫu nhiên

Ví dụ minh họa — tính xác suất từng bước

Xác suất hai người bất kỳ cùng chọn đi khách sạn trong một ngày cho trước:

$$0.01 \times 0.01 = 10^{-4}$$

Xác suất họ chọn *cùng một khách sạn* trong số 10^5 khách sạn:

$$10^{-4}/10^5 = 10^{-9}$$

Xác suất họ ở cùng khách sạn vào **hai ngày khác nhau** (bình phương, vì hai ngày độc lập):

$$(10^{-9})^2 = 10^{-18}$$

Đây là xác suất cho *một* cặp người và *một* cặp ngày cụ thể, với giả định hoàn toàn ngẫu nhiên.

Ví dụ minh họa — số lượng cặp khả nghi kỳ vọng

Số cặp người có thể có: $\binom{10^9}{2} \approx 5 \times 10^{17}$

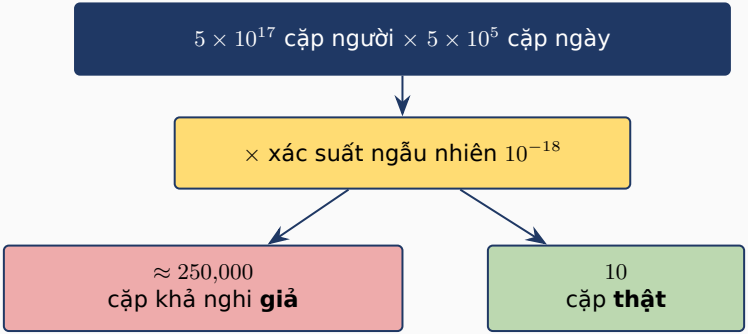
Số cặp ngày có thể có (trong 1.000 ngày): $\binom{1000}{2} \approx 5 \times 10^5$

Số **sự kiện khả nghi** kỳ vọng (hoàn toàn từ dữ liệu ngẫu nhiên):

$$5 \times 10^{17} \times 5 \times 10^5 \times 10^{-18} = 250{,}000$$

Tức là: dù **không có bất kỳ kẻ xấu thật nào**, ta vẫn kỳ vọng tìm thấy khoảng **250.000 cặp người** trông giống như đang "âm mưu" cùng nhau — chỉ vì sự trùng hợp ngẫu nhiên.

Trực quan hóa: mẫu giả áp đảo mẫu thật



Nếu thực sự chỉ có 10 cặp kẻ xấu, cảnh sát sẽ phải điều tra thêm một phần tư triệu cặp vô tội để tìm ra họ

Bài học rút ra

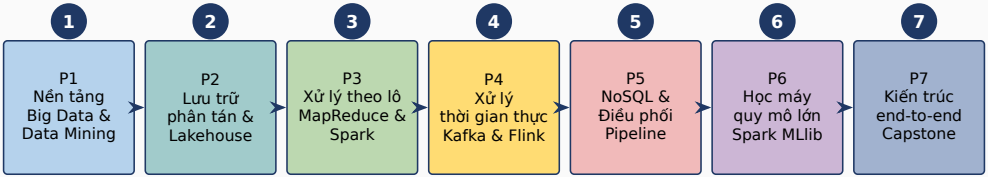
- Nếu thực sự chỉ có **10 cặp** kẻ xấu tồn tại, cảnh sát sẽ phải điều tra **250.000 cặp khác** — tuyệt đại đa số là người vô tội
- Cách tiếp cận "tìm mọi trùng hợp" này **không khả thi** về mặt thực tế lẫn đạo đức
- Nguyên tắc chung: khi tìm sự kiện hiếm (rare event) trong dữ liệu lớn, phải thiết kế tiêu chí tìm kiếm **đủ hẹp/đủ hiếm** để số lượng kỳ vọng từ dữ liệu ngẫu nhiên nhỏ hơn nhiều so với số thực thể thật muốn tìm
- Đây là mối liên hệ trực tiếp với đặc trưng **Veracity** đã học — quy mô dữ liệu lớn (Volume) không tự động đảm bảo độ tin cậy (Veracity) của kết luận rút ra

Ý nghĩa đối với môn học và thực hành nghề nghiệp

- Trước khi học các thuật toán khai phá dữ liệu quy mô lớn ở Phần 3 và Phần 6, cần hiểu: **sức mạnh tính toán không thay thế được tư duy thống kê cẩn trọng**
- Khi báo cáo kết quả khai phá dữ liệu, luôn tự hỏi: "nếu dữ liệu là ngẫu nhiên, tôi có kỳ vọng tìm thấy điều tương tự không?"
- Nguyên lý Bonferroni cũng là lời nhắc nhở về đạo đức dữ liệu: các hệ thống giám sát/dự đoán quy mô lớn (chấm điểm tín dụng, phát hiện gian lận, giám sát an ninh...) cần được thiết kế và kiểm chứng cẩn thận để tránh oan sai hàng loạt

KIẾN THỨC NỀN TẢNG TRƯỚC KHI KHAI PHÁ DỮ LIỆU LỚN

Năm công cụ sẽ xuất hiện lặp lại xuyên suốt môn học



Vì sao cần những kiến thức nền này?

- Trước khi đi vào các thuật toán khai phá dữ liệu cụ thể, có một số công cụ "nền" sẽ được dùng đi dùng lại trong suốt học phần
- Năm khối kiến thức (theo *Mining of Massive Datasets*, Mục 1.3):
  - Độ đo TF.IDF — đo mức độ quan trọng của từ trong văn bản
  - Hash function — nền tảng của phân tán dữ liệu và index
  - Index — cấu trúc dữ liệu để truy xuất nhanh
  - Secondary storage — vì sao thiết kế thuật toán phải "né" đĩa cứng
  - Power laws — quy luật phân bố thường gặp trong dữ liệu thực tế

TF.IDF — vấn đề: từ nào quan trọng trong văn bản?

- Bài toán: phân loại văn bản theo chủ đề thường bắt đầu bằng việc tìm **từ đặc trưng** của mỗi chủ đề
- Trực giác sai lầm: từ xuất hiện **nhiều nhất** trong văn bản là từ quan trọng nhất
- Thực tế: từ xuất hiện nhiều nhất thường là **stop words** ("the", "and", "là", "và"...) — không mang ý nghĩa phân biệt chủ đề
- Từ quan trọng là từ **tương đối hiếm** trên toàn bộ kho văn bản, nhưng **tập trung** vào một số ít văn bản cụ thể

TF.IDF — công thức Term Frequency (TF)

Cho một tập gồm N văn bản. Gọi f_{ij} là số lần từ i xuất hiện trong văn bản j.

Term Frequency của từ i trong văn bản j:

$$TF_{ij} = \frac{f_{ij}}{\max_k f_{kj}}$$

Tức là: chuẩn hóa theo số lần xuất hiện của từ *phổ biến nhất* trong cùng văn bản j. Từ phổ biến nhất trong văn bản có $TF = 1$; các từ khác nhận giá trị phân số.

TF.IDF — công thức Inverse Document Frequency (IDF)

Gọi n_i là số văn bản (trong tổng số N) có chứa từ i.

$$IDF_i = \log_2 \left(\frac{N}{n_i} \right)$$

Điểm số TF.IDF của từ i trong văn bản j:

$$TF.IDF_{ij} = TF_{ij} \times IDF_i$$

Từ càng *hiếm* trên toàn kho (n_i nhỏ) $\Rightarrow$ IDF_i càng lớn. Từ có điểm TF.IDF cao nhất thường là từ đặc trưng nhất cho chủ đề văn bản.

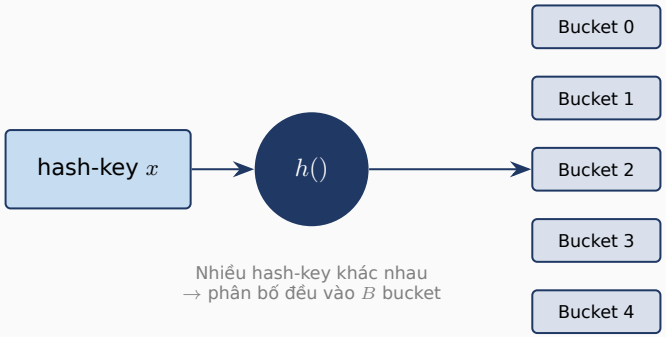
TF.IDF — ví dụ số cụ thể

- Giả sử kho có $2^{20} = 1.048.576$ văn bản, từ w xuất hiện trong $2^{10} = 1024$ văn bản
- $IDF_w = \log_2(2^{20}/2^{10}) = \log_2(2^{10}) = 10$
- Xét văn bản j mà w là từ phổ biến nhất (xuất hiện 20 lần, cũng là max):
  - $TF_{wj} = 1 \Rightarrow$ điểm TF.IDF $= 1 \times 10 = 10$
- Xét văn bản k mà w chỉ xuất hiện 1 lần, trong khi từ phổ biến nhất xuất hiện 20 lần:
  - $TF_{wk} = 1/20 \Rightarrow$ điểm TF.IDF $= 1/20 \times 10 = 0.5$
- $\Rightarrow$ Cùng một từ w, nhưng mức độ "đặc trưng" khác nhau rõ rệt tùy văn bản

TF.IDF — ứng dụng trong Big Data Analytics

- Nền tảng cho **tìm kiếm văn bản** (search engine), phân loại tài liệu, phát hiện spam
- Là bước tiền xử lý phổ biến trước khi đưa văn bản vào các mô hình học máy (sẽ gặp lại ở Phần 6 — Spark MLlib)
- Cũng là nền tảng của kỹ thuật biểu diễn văn bản thành vector để tính **độ tương tự** giữa các tài liệu (liên hệ "Similar Items" đã đề cập)

Hash Functions — khái niệm cơ bản



Hash function nhận một hash-key và trả về một số hiệu bucket (0 đến $B-1$); một hash function tốt phân bố đều dữ liệu vào các bucket

Hash Functions — vì sao chọn B cẩn thận?

- Hash function *h* nhận một *hash-key* và trả về một *bucket number* trong khoảng 0 đến $B-1$
- *Ví dụ phản chứng*: nếu hash-key toàn là số chẵn và $h(x) = x \bmod 10$, thì chỉ các bucket $0, 2, 4, 6, 8$ được dùng — **không phân bố đều**
- Nguyên tắc thực hành: chọn *B* là **số nguyên tố** sẽ giảm nguy cơ phân bố lệch
- Với hash-key không phải số nguyên (chuỗi, bản ghi phức tạp): chuyển đổi về số nguyên bằng mã ASCII/Unicode rồi áp dụng nguyên tắc tương tự

Hash Functions — ví dụ số cụ thể: chọn B thế nào?

Khóa x (mã sinh viên)	$x \bmod 10$	$x \bmod 11$
2002	**2**	0
2004	**4**	2
2006	**6**	4
2008	**8**	6
2010	**0**	8
2012	**2**	10
2014	**4**	1
2016	**6**	3
2018	**8**	5
2020	**0**	7

Hash Functions — nhận xét từ ví dụ trên

- Dữ liệu đầu vào: 10 mã sinh viên, **tất cả đều là số chẵn**, cách đều nhau 2 đơn vị — một tình huống rất thực tế (mã được cấp tuần tự, chẵn/lẻ theo hệ hoặc theo lớp)
- Với $B = 10$ (hợp số): $h(x) = x \bmod 10$
  - Chỉ **5 trên 10 bucket** được dùng: $\{0, 2, 4, 6, 8\}$ — mỗi bucket nhận đúng **2 khóa** (collision)
  - Các bucket $\{1, 3, 5, 7, 9\}$ **không bao giờ** được dùng, dù dữ liệu trông như "ngẫu nhiên"
- Với $B = 11$ (số nguyên tố): $h(x) = x \bmod 11$
  - **10 trên 11 bucket** được dùng, **không có collision nào** — phân bố gần như hoàn hảo
- $\Rightarrow$ Bài học: chọn B là **số nguyên tố** giúp "phá vỡ" các quy luật số học ẩn trong khóa (ở đây là "luôn luôn chẵn"), tránh phân bố lệch dù ta không biết trước đặc điểm của dữ liệu

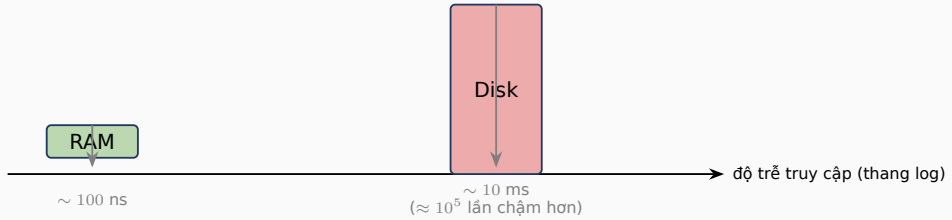
Hash Functions — vì sao quan trọng cho Big Data?

- Là cơ chế cốt lõi để **phân tán dữ liệu đồng đều** trên nhiều máy trong cluster — sẽ gặp lại ở Phần 2 (HDFS block placement) và Phần 3 (MapReduce partitioning, shuffle)
- Là nền tảng của các cấu trúc dữ liệu xác suất tiết kiệm bộ nhớ (Bloom filter, HyperLogLog) dùng trong xử lý luồng dữ liệu — sẽ gặp ở Phần 4
- Là thành phần chính của kỹ thuật **Locality-Sensitive Hashing (LSH)** để tìm các mục tương tự trên dữ liệu khổng lồ (Chương 3 của *Mining of Massive Datasets*)

Indexes — cấu trúc dữ liệu để truy xuất nhanh

- **Index** là cấu trúc dữ liệu giúp truy xuất hiệu quả các bản ghi theo giá trị của một hoặc nhiều trường
- *Ví dụ*: có tệp các bộ ba (tên, địa chỉ, số điện thoại) và một index trên trường số điện thoại — cho một số điện thoại, ta tìm ngay được bản ghi tương ứng mà không cần duyệt toàn bộ tệp
- Cách hiện thực đơn giản nhất: dùng **hash table** làm index — trường cần tra cứu chính là hash-key
- Nếu số lượng bucket B tương đương số bản ghi, mỗi bucket chỉ chứa rất ít bản ghi $\Rightarrow$ tìm kiếm rất nhanh

Secondary Storage — khoảng cách tốc độ RAM và Disk

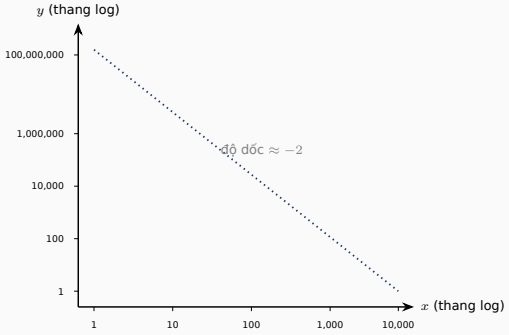


Đọc một block từ đĩa mất khoảng 10ms — chậm hơn khoảng 10^5 lần so với đọc một từ (word) từ RAM

Secondary Storage — hệ quả cho thiết kế thuật toán

- Đĩa được tổ chức thành các **block** (đơn vị nhỏ nhất di chuyển giữa đĩa và RAM) — ví dụ Windows dùng block $2^{16} = 65.536$ byte
- Thời gian **truy cập** một block (di chuyển đầu đọc + chờ xoay đĩa) khoảng 10ms — trong khi đọc RAM chỉ mất vài chục **nano**giây
- Hệ quả: nếu thuật toán truy cập đĩa **ngẫu nhiên, từng byte một**, thời gian di chuyển dữ liệu sẽ lớn hơn nhiều lần thời gian tính toán thực sự
- Nguyên tắc thiết kế Big Data: tổ chức dữ liệu để đọc **tuần tự theo block/cylinder**, tránh truy cập ngẫu nhiên — đây là lý do vì sao HDFS/Spark ưu tiên đọc tuần tự khối lớn thay vì random I/O nhỏ lẻ

Power Laws — quy luật phân bố thường gặp



Trên trục log-log, một power law thể hiện thành đường thẳng: $\log_{10} y = b + a \log_{10} x$, *tức* $y = cx^a$. *Ví dụ trong hình: phân bố số liên kết-đến (in-links) của các trang Web, độ dốc* $a \approx -2$

Power Laws — ví dụ số cụ thể: dữ liệu tần suất từ theo hạng

Hạng x (rank)	**Tần suất y**	$\log_{10} x$	$\log_{10} y$
1	100.000	0	5
10	10.000	1	4
100	1.000	2	3
1.000	100	3	2
10.000	10	4	1

Power Laws — đọc bảng số liệu trên

- Giả lập: xếp hạng các từ trong một kho văn bản theo tần suất xuất hiện — hạng 1 là từ phổ biến nhất (ví dụ "the", "và")
- Mỗi khi hạng x tăng gấp **10 lần**, tần suất y lại giảm đúng **10 lần** — một quy luật rất đều đặn, không hề "ngẫu nhiên"
- Hai cột bên phải ($\log_{10} x$, $\log_{10} y$) chính là tọa độ của các điểm này trên đồ thị log-log ở slide trước — đây là cách "giải mã" một power law từ số liệu thô

Power Laws — tính độ dốc a từ hai điểm dữ liệu

Chọn hai điểm bất kỳ trên đường thẳng log-log, (x_1, y_1) và (x_2, y_2):

$$a = \frac{\log_{10} y_2 - \log_{10} y_1}{\log_{10} x_2 - \log_{10} x_1}$$

Áp dụng với điểm đầu $(x_1, y_1) = (1,\ 100.000)$ và điểm cuối $(x_2, y_2) = (10.000,\ 10)$ từ bảng trên:

$$a = \frac{\log_{10} 10 - \log_{10} 100.000}{\log_{10} 10.000 - \log_{10} 1} = \frac{1-5}{4-0} = -1$$

Vậy $y = c \cdot x^{-1}$ với $c = 100.000$ (giá trị y tại $x = 1$) — đây chính xác là dạng của **Zipf's Law**.

Kiểm tra chéo với một cặp điểm khác, ví dụ $(10,\ 10.000)$ và $(1.000,\ 100)$: $a = (2-4)/(3-1) = -1$ — khớp, xác nhận dữ liệu tuân theo một power law "sạch".

Power Laws — ý nghĩa của kết quả vừa tính

- Độ dốc $a = -1$: mỗi khi hạng x tăng gấp 10 lần, tần suất y giảm đúng 10 lần — đây là "dấu vân tay" đặc trưng của Zipf's Law ($a \approx -1$)
- So sánh với đồ thị ở slide trước (độ dốc $a \approx -2$, ví dụ in-links của trang Web): độ dốc **càng âm sâu** thì phân bố **càng lệch mạnh** — khối lượng càng dồn về rất ít phần tử đứng đầu bảng xếp hạng
- Kỹ thuật "chọn hai điểm bất kỳ, tính tỉ số log" là cách nhanh nhất để kiểm tra một tập dữ liệu thực tế có tuân theo power law hay không — không cần công cụ thống kê phức tạp, chỉ cần vẽ log-log và đo độ dốc

Power Laws — Zipf's Law và hiệu ứng Matthew

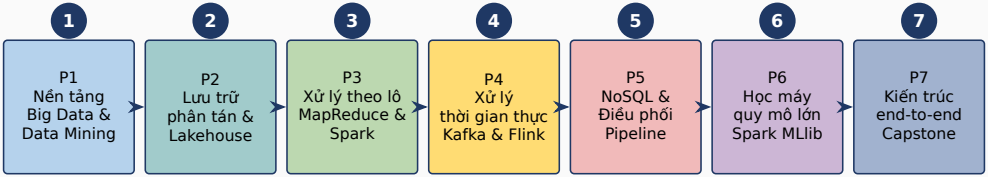
- Zipf's Law
  - Xếp hạng từ theo tần suất xuất hiện — tần suất tuân theo power law với độ dốc $a \approx -1$ ($y \approx cx^{-1}$), đúng như ví dụ số vừa tính ở trên
  - Cũng áp dụng cho dân số các thành phố, doanh số sách bán chạy nhất...
- Hiệu ứng Matthew ("giàu càng giàu")
  - Trang Web có nhiều liên kết đến càng dễ được tìm thấy và liên kết thêm — số liên kết-đến (in-links) tuân theo power law với độ dốc khoảng -2.1
  - Sách bán chạy trên Amazon càng được quảng bá nhiều hơn $\Rightarrow$ bán càng chạy hơn
- Ý nghĩa cho Big Data: rất nhiều hiện tượng thực tế **không phân bố đều (uniform)** mà tập trung ở một số ít phần tử — ảnh hưởng trực tiếp đến thiết kế hệ thống (data skew khi phân tán, long tail trong hệ gợi ý ở Phần 6)

Năm khối kiến thức nền — sẽ gặp lại ở đâu?

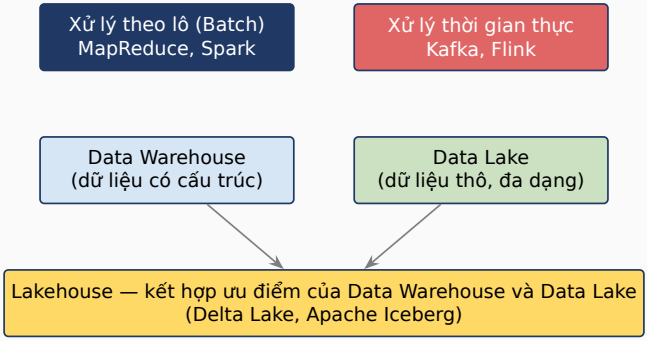
Kiến thức nền	Áp dụng ở phần nào của roadmap
TF.IDF	P6 (xử lý văn bản trong MLlib), tìm kiếm/gợi ý
Hash Functions	P2 (HDFS), P3 (MapReduce shuffle), P4 (Bloom filter)
Indexes	P2, P5 (NoSQL key-value/document store)
Secondary Storage	P2 (thiết kế HDFS block), P3 (tối ưu I/O của Spark)
Power Laws	P5/P6 (data skew), gợi ý sản phẩm (long tail)

TOÀN CẢNH HỆ SINH THÁI CÔNG NGHỆ BIG DATA HIỆN ĐẠI

Hai trục phân loại chính: cách xử lý và cách lưu trữ



Toàn cảnh hệ sinh thái công nghệ Big Data



Hai trục phân loại chính: cách xử lý (batch/streaming) và cách lưu trữ (warehouse/lake/lakehouse) — sẽ được trình bày chi tiết ở Phần 2–4

Xử lý theo lô (Batch) vs. Thời gian thực (Streaming)

Batch (theo lô)

- Xử lý một khối dữ liệu **đã tích lũy đủ lớn**, chạy định kỳ (ví dụ mỗi đêm)
- Thông lượng (throughput) cao, độ trễ (latency) lớn — có thể vài phút đến vài giờ
- Công cụ tiêu biểu: MapReduce, Apache Spark (batch mode)

Streaming (thời gian thực)

- Xử lý **từng sự kiện** ngay khi nó phát sinh, gần như tức thời
- Độ trễ thấp (mili-giây đến giây), nhưng throughput/sự kiện thường thấp hơn batch
- Công cụ tiêu biểu: Apache Kafka, Apache Flink, Spark Structured Streaming

Data Warehouse — kho dữ liệu truyền thống

- Lưu trữ dữ liệu **đã được làm sạch, có cấu trúc**, theo schema xác định trước ("schema-on-write")
- Tối ưu cho truy vấn phân tích (OLAP), báo cáo kinh doanh (BI)
- **Hạn chế**: khó lưu dữ liệu phi cấu trúc (ảnh, log, văn bản tự do); chi phí lưu trữ cao; thiếu linh hoạt khi schema thay đổi

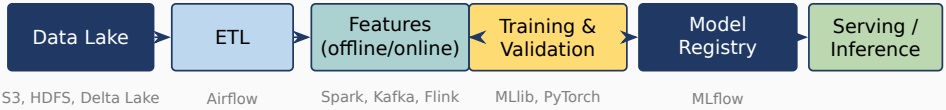
Data Lake — hồ dữ liệu

- Lưu trữ dữ liệu **thô, ở dạng gốc**, không cần định nghĩa schema trước ("schema-on-read") — chấp nhận mọi định dạng (Variety cao)
- Chi phí lưu trữ rẻ hơn nhiều (thường trên Cloud Object Storage: Amazon S3, GCS, Azure Blob)
- **Hạn chế**: nếu không quản lý tốt dễ trở thành "data swamp" — dữ liệu hỗn loạn, khó tin cậy (Veracity thấp), thiếu tính năng giao dịch (transaction) như ACID

Lakehouse — kết hợp ưu điểm của cả hai

- Ý tưởng: xây dựng lớp quản lý giao dịch (ACID), lược đồ, và tối ưu truy vấn **ngay trên** nền tảng lưu trữ giá rẻ của Data Lake
- Công nghệ tiêu biểu: **Delta Lake, Apache Iceberg, Apache Hudi** — sẽ thực hành trực tiếp ở Tuần 3 (Phần 2)
- Kết quả: vừa linh hoạt & rẻ như Data Lake, vừa đáng tin cậy & dễ truy vấn như Data Warehouse
- Đây là kiến trúc lưu trữ chủ đạo của các hệ thống Big Data hiện đại (2020s trở đi)

Data Mining/ML trong pipeline hiện đại — nhìn lại



Mỗi khối trong pipeline tương ứng với một phần trong roadmap: Data Lake/Lakehouse (P2), ETL/Streaming (P3–P5), Training (P6)

Vai trò của từng công cụ trong bức tranh lớn

Công cụ	Vai trò	Phần liên quan
HDFS / S3 / Delta Lake	Lưu trữ phân tán, lakehouse	P2
Hadoop MapReduce / Spark	Xử lý dữ liệu theo lô	P3
Kafka / Flink	Xử lý dữ liệu thời gian thực	P4
MongoDB / Redis / Airflow	NoSQL & điều phối pipeline	P5
Spark MLlib	Học máy quy mô lớn	P6

Case study: pipeline Big Data cho một sàn TMĐT



- **Ingestion**: log click/hành vi đổ liên tục vào Kafka (streaming) và vào Data Lake (batch)
- **Lưu trữ**: dữ liệu thô lưu trên Data Lake, được tổ chức lại thành bảng Lakehouse (Delta/Iceberg)
- **Xử lý**: Spark tính toán các đặc trưng (feature) theo lô mỗi đêm, Flink xử lý sự kiện realtime cho gợi ý tức thời
- **Phục vụ**: mô hình gợi ý sản phẩm (Spark MLlib) chạy trên các đặc trưng đã tính

Vì sao một hệ thống thực tế cần cả Batch lẫn Streaming?

- **Batch** phù hợp cho phân tích sâu, huấn luyện mô hình phức tạp trên toàn bộ lịch sử dữ liệu — độ chính xác cao, không cần phản hồi ngay
- **Streaming** phù hợp cho phản ứng tức thời — ví dụ cập nhật gợi ý ngay sau một cú click, phát hiện gian lận khi giao dịch vừa xảy ra
- Phần lớn hệ thống Big Data thực tế **kết hợp cả hai** — mô hình kiến trúc **Lambda** và **Kappa** (sẽ học chi tiết ở Phần 7) chính là các khung thiết kế chuẩn hóa cho sự kết hợp này

Cầu nối sang phần tiếp theo



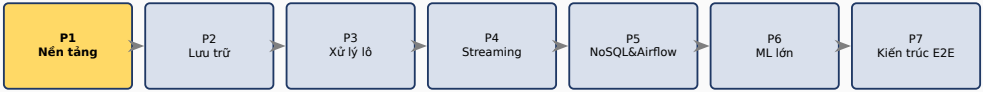
*Đã có bức tranh toàn cảnh về **hạ tầng** công nghệ Big Data. Câu hỏi tiếp theo:*

- Trên nền hạ tầng đó, chúng ta thực sự **làm gì** với dữ liệu để rút ra tri thức?
- → Quay lại với **bốn kỹ thuật khai phá dữ liệu nền tảng** đã giới thiệu sơ lược ở đầu buổi — giờ ta đi sâu vào từng kỹ thuật

PHẦN 1

BỐN KỸ THUẬT KHAI PHÁ DỮ LIỆU NỀN TẢNG

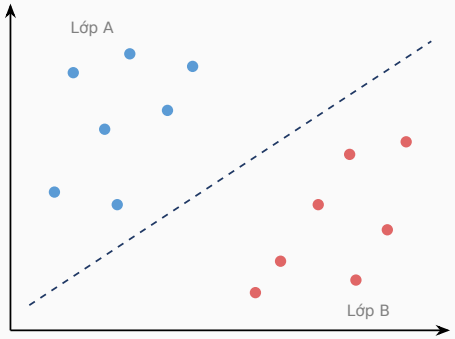
Classification, Clustering, Association Rules, Regression



Khung phân loại chung: có giám sát vs. không giám sát

- Học có giám sát (supervised)
  - Có tập dữ liệu **đã biết nhãn/kết quả** để huấn luyện
  - Mục tiêu: học một hàm ánh xạ từ đầu vào sang đầu ra đã biết
- Học không giám sát (unsupervised)
  - **Không** có nhãn — chỉ có dữ liệu thô
  - Mục tiêu: tự khám phá cấu trúc ẩn trong dữ liệu
- Bốn kỹ thuật sắp học chia làm hai nhóm: Classification & Regression (có giám sát), Clustering & Association Rules (không giám sát)

Classification (Phân loại)



Tìm một ranh giới (boundary) phân tách các lớp dữ liệu đã biết nhãn, dùng để dự đoán nhãn cho dữ liệu mới

Classification — định nghĩa & quy trình

- Gán một **nhãn/lớp đã biết trước** cho dữ liệu mới, dựa trên dữ liệu đã gán nhãn (học có giám sát)
- Quy trình chuẩn:
  - 1. Thu thập tập huấn luyện (training set) đã có nhãn
  - 2. Huấn luyện mô hình học ranh giới phân lớp
  - 3. Dùng mô hình để dự đoán nhãn cho dữ liệu mới, chưa có nhãn
- *Ví dụ kinh điển*: phân loại email là spam hay không spam, dựa trên các email đã được gán nhãn trước đó

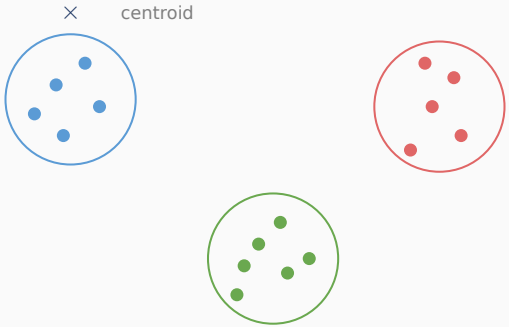
Classification — thuật toán tiêu biểu

- **Decision Tree**: chia dữ liệu theo các luật "nếu...thì..." liên tiếp, dễ diễn giải
- **Naive Bayes**: dựa trên định lý Bayes với giả định các đặc trưng độc lập nhau
- **Logistic Regression**: mô hình tuyến tính cho xác suất thuộc về một lớp
- **Support Vector Machine (SVM)**: tìm ranh giới phân lớp có "biên an toàn" lớn nhất
- Cả bốn thuật toán này đều có phiên bản chạy phân tán trong Spark MLlib — sẽ thực hành ở Phần 6

Classification — ví dụ ứng dụng thực tế

Lĩnh vực	Bài toán phân loại
Email/Bảo mật	Spam hay không spam? Tấn công mạng hay bình thường?
Ngân hàng	Khách hàng có rủi ro tín dụng cao hay thấp?
Y tế	Ảnh X-quang có dấu hiệu bệnh lý hay không?
TMĐT	Khách hàng có rời bỏ (churn) hay không?

Clustering (Phân cụm)



Nhóm các điểm dữ liệu gần nhau vào cùng một cụm, mỗi cụm được tóm tắt bằng tâm cụm (centroid) — không cần nhãn trước

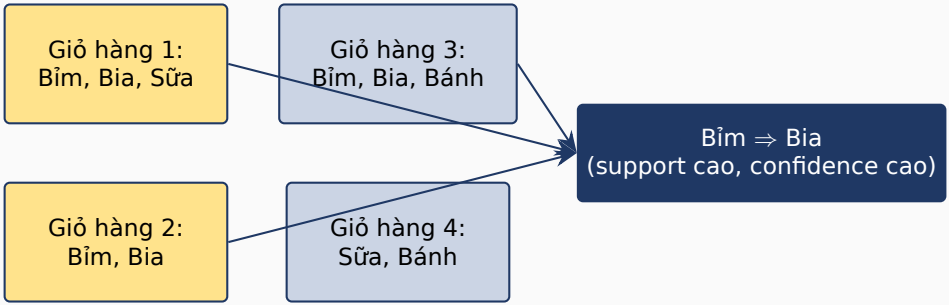
Clustering — định nghĩa & thuật toán tiêu biểu

- Nhóm các đối tượng có đặc điểm **tương tự** vào cùng một cụm, **không cần** nhãn trước (học không giám sát)
- **K-Means**: chọn trước k tâm cụm, lặp lại việc gán điểm vào tâm gần nhất rồi cập nhật lại tâm — thuật toán phổ biến nhất, sẽ học chi tiết ở Phần 6
- **Hierarchical Clustering**: xây dựng cây phân cụm từ dưới lên (gộp dần các cụm gần nhau) hoặc từ trên xuống
- Nguồn tham khảo chi tiết: *Mining of Massive Datasets*, Chương 7

Clustering — ví dụ ứng dụng

- **Phân khúc khách hàng** theo hành vi mua sắm để cá nhân hóa chiến dịch marketing
- **Nén/tóm tắt dữ liệu**: thay vì lưu toàn bộ điểm dữ liệu, chỉ cần lưu tâm cụm và độ lệch
- Nhắc lại case study **John Snow**: về bản chất, ông đã thực hiện một phép "phân cụm thủ công" để tìm ra nguồn gây dịch tả — nguyên lý không đổi dù làm bằng tay hay bằng thuật toán K-Means trên hàng triệu điểm dữ liệu

Association Rules (Luật kết hợp)



Mô hình "giỏ hàng" (market-basket): tìm luật "nếu mua A thì thường mua B"

Association Rules — mô hình Market-Basket

- Dữ liệu được xem như một tập các **giỏ hàng** (basket), mỗi giỏ chứa một tập nhỏ các mục (item)
- Tìm các mối quan hệ "nếu... thì..." xuất hiện thường xuyên
- Ví dụ kinh điển trong ngành bán lẻ: khách mua bỉm thường mua kèm bia (market-basket analysis)
- Nguồn tham khảo chi tiết: *Mining of Massive Datasets*, Chương 6 — thuật toán A-Priori

Association Rules — công thức Support & Confidence

Cho luật $I \Rightarrow j$ (tập mục I suy ra mục j):

Support — tỷ lệ giỏ hàng chứa *cả* I và j trên tổng số giỏ hàng:

$$\text{support}(I \Rightarrow j) = \frac{\text{số giỏ chứa } I \cup \{j\}}{\text{tổng số giỏ hàng}}$$

Confidence — trong số giỏ đã chứa I, tỷ lệ giỏ *cũng* chứa j:

$$\text{confidence}(I \Rightarrow j) = \frac{\text{số giỏ chứa } I \cup \{j\}}{\text{số giỏ chứa } I}$$

Một luật được xem là *mạnh* khi cả support và confidence đều đủ cao.

Association Rules — ví dụ số minh họa

- Giả sử có 4 giỏ hàng: {Bỉm,Bia,Sữa}, {Bỉm,Bia}, {Bỉm,Bia,Bánh}, {Sữa,Bánh}
- Xét luật **Bỉm** $\Rightarrow$ **Bia**: có 3/4 giỏ chứa Bỉm, và cả 3 giỏ đó đều chứa Bia
- support $= 3/4 = 0.75$ — confidence $= 3/3 = 1.0$
- $\Rightarrow$ Đây là một luật **rất mạnh**: hầu như ai mua Bỉm cũng mua Bia

Association Rules — thuật toán A-Priori: động lực

- Thách thức: với *n* mặt hàng, số tập con có thể có là 2^n — với vài nghìn mặt hàng, con số này **vượt xa** khả năng đếm trực tiếp trên toàn bộ dữ liệu
- *Ví dụ*: chỉ 1.000 mặt hàng đã có $\binom{1000}{2} \approx 500.000$ cặp cần đếm nếu kiểm tra toàn bộ — và đó mới là các **cặp**, chưa tính bộ ba, bộ bốn...
- Nguyên lý **Monotonicity** (đơn điệu) — chìa khóa để cắt giảm:
  - Nếu một tập mục *I* **không thường xuyên** (không đạt ngưỡng support), thì **mọi tập lớn hơn chứa** *I* cũng chắc chắn không thường xuyên
  - Suy ra chiều ngược lại: một tập mục **chỉ có thể** thường xuyên nếu **tất cả** tập con của nó đều đã thường xuyên
- A-Priori (Agrawal & Srikant, 1994) khai thác nguyên lý này để chỉ đếm những ứng viên "có triển vọng", thay vì đếm toàn bộ 2^n tổ hợp

A-Priori — ví dụ số: mở rộng dữ liệu giỏ hàng

Giỏ hàng	Các mục
B_1	Bỉm, Bia, Sữa
B_2	Bỉm, Bia
B_3	Bỉm, Bia, Bánh
B_4	Sữa, Bánh
B_5	Bỉm, Sữa, Trứng
B_6	Bia, Bánh, Trứng

A-Priori — ví dụ số: dữ liệu & ngưỡng hỗ trợ

- Mở rộng ví dụ 4 giỏ hàng đã dùng ở slide trước, thêm B_5, B_6 — tổng cộng **6 giỏ hàng**, 5 mặt hàng khác nhau: Bỉm, Bia, Sữa, Bánh, Trứng
- Chọn ngưỡng hỗ trợ tối thiểu: $s = 3$ giỏ hàng (tương đương support $\geq 3/6 = 50\%$)
- Mục tiêu: tìm mọi tập mục **thường xuyên** (frequent itemset) — xuất hiện trong **ít nhất 3** giỏ hàng, theo từng cấp $k = 1, 2, 3, \ldots$

A-Priori — Pass 1: đếm từng mục đơn ($C_1 \to L_1$)

Mục	**Số giỏ chứa (count)**	$\geq s = 3$**?**
Bỉm	4 (B_1, B_2, B_3, B_5)	**Có**
Bia	4 (B_1, B_2, B_3, B_6)	**Có**
Sữa	3 (B_1, B_4, B_5)	**Có**
Bánh	3 (B_3, B_4, B_6)	**Có**
Trứng	2 (B_5, B_6)	Không

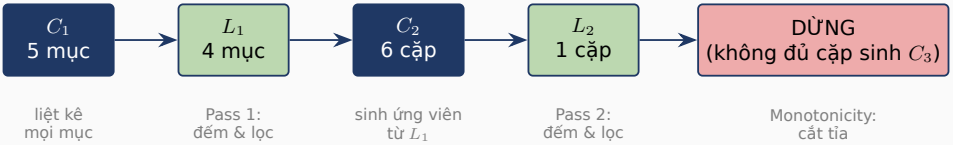
A-Priori — sinh ứng viên cặp C_2 từ L_1

- $L_1 = \{\text{Bỉm, Bia, Sữa, Bánh}\}$ — 4 mục thường xuyên (Trứng bị loại vì count $= 2 < 3$)
- C_2: ghép **mọi cặp** mục trong L_1 — $\binom{4}{2} = 6$ ứng viên: {Bỉm,Bia}, {Bỉm,Sữa}, {Bỉm,Bánh}, {Bia,Sữa}, {Bia,Bánh}, {Sữa,Bánh}
- Nhờ Monotonicity, ta **không cần** xét các cặp có chứa Trứng (ví dụ {Bỉm,Trứng}) — vì Trứng đã không thường xuyên ở cấp 1, mọi cặp chứa Trứng chắc chắn cũng không thường xuyên
- $\Rightarrow$ Từ $\binom{5}{2} = 10$ cặp có thể có, chỉ còn phải **đếm 6 cặp** — đã cắt giảm 40% khối lượng công việc ngay từ cấp 2

A-Priori — Pass 2: đếm từng cặp ứng viên ($C_2 \to L_2$)

Cặp ứng viên	**Số giỏ chứa cả hai**	$\geq s = 3$**?**
{Bỉm, Bia}	3 (B_1, B_2, B_3)	**Có**
{Bỉm, Sữa}	2 (B_1, B_5)	Không
{Bỉm, Bánh}	1 (B_3)	Không
{Bia, Sữa}	1 (B_1)	Không
{Bia, Bánh}	2 (B_3, B_6)	Không
{Sữa, Bánh}	1 (B_4)	Không

A-Priori — toàn cảnh quy trình xử lý theo từng cấp



Mỗi cấp k cần đúng một lượt quét dữ liệu (Pass k); tập ứng viên C_k luôn được sinh ra chỉ từ L_{k-1} đã biết là thường xuyên

A-Priori — vì sao dừng lại sau Pass 2?

- $L_2 = \{\{\text{Bỉm, Bia}\}\}$ — **chỉ một** cặp thường xuyên duy nhất
- Để sinh một ứng viên bộ ba C_3 hợp lệ, cần **ít nhất hai** cặp trong L_2 **chung nhau một mục** để ghép lại (ví dụ {Bỉm,Bia} ghép với {Bỉm,Sữa} mới ra được {Bỉm,Bia,Sữa})
- Vì L_2 chỉ có đúng 1 phần tử — **không thể ghép được cặp nào cả** $\Rightarrow C_3 = \varnothing$ — thuật toán **dừng ngay**, không cần thêm lượt quét nào nữa
- Đây chính là sức mạnh của Monotonicity: ta **biết chắc** {Bỉm,Bia,Sữa} không thể thường xuyên (vì tập con {Bỉm,Sữa} và {Bia,Sữa} đã không thường xuyên) mà **không cần đếm nó** trên dữ liệu gốc

A-Priori — thuật toán tổng quát (đa cấp k)

- Bước 1: đặt $k = 1$; C_1 = tất cả các mục đơn xuất hiện trong dữ liệu
- Bước 2: quét **toàn bộ** dữ liệu một lượt, đếm số giỏ chứa mỗi ứng viên trong C_k; giữ lại các ứng viên có count $\geq s$ để tạo L_k
- Bước 3: nếu $L_k = \varnothing$, **DỪNG**; ngược lại, sinh C_{k+1} bằng cách ghép các cặp itemset trong L_k có chung $k-1$ mục, rồi loại bỏ mọi ứng viên có tập con không thường xuyên (Monotonicity)
- Bước 4: $k \leftarrow k+1$, quay lại Bước 2
- $\Rightarrow$ Mỗi lượt lặp cần **một lượt quét toàn bộ dữ liệu** (I/O trên đĩa) — liên hệ trực tiếp với bài học **Secondary Storage**: càng ít lượt quét, thuật toán càng hiệu quả trên dữ liệu lớn

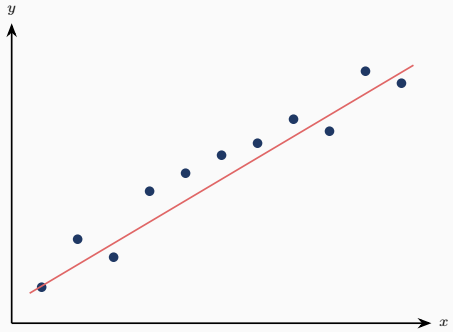
A-Priori — kiểm chứng ví dụ trên bằng Python

```
from itertools import combinations
from collections import Counter
baskets = [
    {"Bỉm","Bia","Sữa"}, {"Bỉm","Bia"}, {"Bỉm","Bia","Bánh"},
    {"Sữa","Bánh"}, {"Bỉm","Sữa","Trứng"}, {"Bia","Bánh","Trứng"},
]
s = 3  # nguong support toi thieu
def count(cands):
    c = Counter()
    for b in baskets:
        for cand in cands:
            if cand <= b: c[frozenset(cand)] += 1
    return c
items = set().union(*baskets)
L1 = {k: v for k, v in count([{i} for i in items]).items() if v >= s}
print("L1:", L1)
C2 = [a | b for a, b in combinations(L1.keys(), 2)]
L2 = {k: v for k, v in count(C2).items() if v >= s}
print("L2:", L2)  # -> chi con {Bim,Bia}: 3, khop voi tinh tay
```

A-Priori — vượt ra ngoài ví dụ nhỏ này

- Ví dụ trên chỉ 6 giỏ hàng, 5 mặt hàng — đủ nhỏ để đếm bằng tay/Python thuần; với hàng triệu giỏ hàng và hàng chục nghìn mặt hàng, mỗi lượt quét (Pass) phải chạy **phân tán** (MapReduce/Spark — Phần 3 & 6)
- Các biến thể tối ưu hơn: **PCY (Park-Chen-Yu)** dùng thêm một bảng băm để lọc bớt ứng viên C_2 ngay trong Pass 1; **Multistage**, **Multihash** giảm tiếp số lượng ứng viên giả (false positive) trong bảng băm
- Nguồn tham khảo chi tiết đầy đủ (chứng minh, độ phức tạp, các biến thể): *Mining of Massive Datasets*, Chương 6

Regression (Hồi quy)



Tìm mối quan hệ (thường là một đường/mặt) khớp tốt nhất với dữ liệu, dùng để dự đoán giá trị số liên tục

Regression — định nghĩa & công thức cơ bản

Dự đoán một giá trị số **liên tục** dựa trên các biến đầu vào (học có giám sát).

Hồi quy tuyến tính đơn giản (1 biến):

$$y = \beta_0 + \beta_1 x + \varepsilon$$

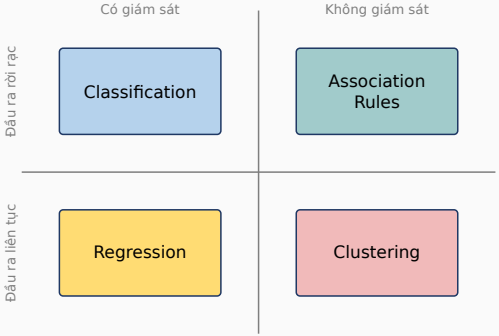
trong đó β_0, β_1 là các tham số được ước lượng sao cho tổng bình phương sai số $\sum (y_i - \hat{y}_i)^2$ là nhỏ nhất — phương pháp *least squares*.

Với nhiều biến đầu vào, mô hình mở rộng thành hồi quy tuyến tính đa biến — nền tảng toán học cho nhiều mô hình học máy phức tạp hơn ở Phần 6.

Regression — ví dụ ứng dụng

- Dự đoán **doanh thu** tháng tới dựa trên dữ liệu bán hàng lịch sử và yếu tố mùa vụ
- Dự đoán **giá nhà** dựa trên diện tích, vị trí, số phòng...
- Dự đoán **điểm số** sinh viên dựa trên số giờ tự học, điểm danh, điểm giữa kỳ
- Trong Big Data Analytics, hồi quy thường được huấn luyện trên hàng triệu bản ghi — cần các thuật toán tối ưu song song hóa được (Stochastic Gradient Descent — sẽ gặp lại ở Phần 6)

Bốn kỹ thuật trên cùng một khung nhìn



Phân loại theo hai trục: có/không giám sát, và đầu ra rời rạc/liên tục

So sánh 4 kỹ thuật khai phá dữ liệu

Kỹ thuật	**Loại học**	**Đầu ra**	**Thuật toán/công thức chính**
Classification	Có giám sát	Nhãn rời rạc	Decision Tree, SVM, Logistic Reg.
Clustering	Không giám sát	Nhóm/cụm	K-Means
Association Rules	Không giám sát	Luật kết hợp	Support, Confidence, A-Priori
Regression	Có giám sát	Giá trị số liên tục	$y = \beta_0 + \beta_1 x$, Least Squares

Case study tổng hợp: áp dụng cả 4 kỹ thuật cho một sàn TMĐT



- **Classification**: dự đoán khách hàng có rời bỏ (churn) hay không
- **Clustering**: phân khúc khách hàng theo hành vi mua sắm
- **Association rules**: gợi ý "sản phẩm thường mua kèm"
- **Regression**: dự đoán doanh thu theo mùa vụ
- Một hệ thống thực tế thường kết hợp **nhiều** kỹ thuật cùng lúc, không chỉ một

Thách thức khi khai phá dữ liệu ở quy mô lớn

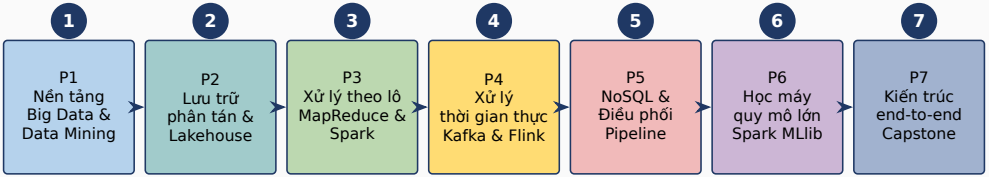
- Thuật toán data mining truyền thống thường giả định dữ liệu **nằm gọn trong bộ nhớ** một máy — không còn đúng ở quy mô Big Data (nhắc lại Secondary Storage đã học)
- Cần các phiên bản thuật toán **phân tán** chạy song song trên nhiều máy — chủ đề của Phần 3 (MapReduce/Spark) và Phần 6 (Spark MLlib)
- **Classification/Clustering** ở quy mô lớn: cần thuật toán như CURE, BFR (biến thể phân tán của K-Means) — MMDS Chương 7
- Chất lượng dữ liệu (Veracity) ảnh hưởng trực tiếp đến độ tin cậy kết quả khai phá — "garbage in, garbage out"

Bốn kỹ thuật sẽ "phóng to" ở đâu trong roadmap?

Kỹ thuật	Phần liên quan trong roadmap
Classification, Regression	P6 — Spark MLlib (huấn luyện phân tán)
Clustering	P6 — K-Means phân tán trên Spark MLlib
Association Rules	P5 — kết hợp NoSQL lưu trữ giao dịch quy mô lớn
Cả 4 kỹ thuật	P7 — tích hợp vào kiến trúc ứng dụng end-to-end

PHÂN LOẠI MỘT BÀI TOÁN KHAI PHÁ DỮ LIỆU

Khung phân loại theo đặc điểm dữ liệu và mô hình tính toán (theo Stanford CS246)



Vì sao cần một khung phân loại?

- Trước khi chọn thuật toán, một câu hỏi quan trọng không kém là: **bài toán này thuộc dạng gì?**
- Cùng một kỹ thuật (ví dụ Classification) có thể cần cách tiếp cận hoàn toàn khác nhau tùy đặc điểm dữ liệu và ràng buộc tính toán
- Stanford CS246 đề xuất phân loại bài toán theo hai trục: **đặc điểm dữ liệu** và **mô hình tính toán** — đây chính là kỹ năng cần rèn ở phần thực hành hôm nay

Trục 1 — Đặc điểm dữ liệu

- Dữ liệu nhiều chiều (high-dimensional)
  - Mỗi điểm dữ liệu có rất nhiều thuộc tính/đặc trưng — ví dụ vector đặc trưng người dùng, văn bản dạng TF.IDF
- Dữ liệu dạng đồ thị (graph)
  - Các thực thể có **quan hệ** với nhau — ví dụ mạng xã hội, mạng liên kết Web
- Dữ liệu vô hạn/liên tục (infinite/streaming)
  - Dữ liệu **không bao giờ dừng lại**, không thể lưu hết — ví dụ luồng log, luồng cảm biến IoT
- Dữ liệu có nhãn (labeled)
  - Có sẵn nhãn/kết quả đúng để huấn luyện mô hình có giám sát

Trục 2 — Mô hình tính toán phù hợp

- MapReduce / xử lý phân tán theo lô
  - Phù hợp khi dữ liệu **lớn nhưng tĩnh** (đã thu thập xong), cần xử lý song song trên nhiều máy — chủ đề Phần 3
- Streams & online algorithms
  - Phù hợp khi dữ liệu **liên tục phát sinh**, không thể lưu hết, cần xử lý ngay khi đến — chủ đề Phần 4
- Single-machine in-memory
  - Phù hợp khi dữ liệu (sau khi lấy mẫu/tóm tắt) đủ nhỏ để **vừa trong RAM một máy** — nhanh và đơn giản, không cần hạ tầng phân tán

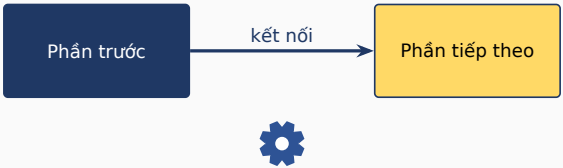
Ví dụ áp dụng khung phân loại

Bài toán	Đặc điểm dữ liệu	Mô hình tính toán phù hợp
Lọc email spam (huấn luyện 1 lần)	Có nhãn, nhiều chiều	Single-machine hoặc MapReduce
Phát hiện gian lận thẻ tín dụng	Có nhãn, luồng liên tục	Streams / online algorithms
Phát hiện cộng đồng mạng xã hội	Dạng đồ thị, khối lượng lớn	MapReduce / xử lý đồ thị phân tán
Đếm số khách truy cập duy nhất/ngày	Luồng vô hạn	Streams (thuật toán xác suất)

Quy trình 3 bước để phân loại một bài toán mới

- Bước 1 — Xác định kỹ thuật khai phá dữ liệu cần dùng
  - Đây là bài toán Classification, Clustering, Association Rules, hay Regression?
- Bước 2 — Xác định đặc điểm dữ liệu
  - Dữ liệu có nhãn không? Có cấu trúc đồ thị không? Là dữ liệu tĩnh hay luồng vô hạn?
- Bước 3 — Chọn mô hình tính toán phù hợp
  - Dữ liệu có vừa trong RAM một máy không? Nếu không, cần MapReduce hay xử lý streaming?
- Quy trình này sẽ được áp dụng trực tiếp trong bài tập phần thực hành

Kết nối sang phần thực hành



Toàn bộ lý thuyết vừa học — 5V, giới hạn Bonferroni, TF.IDF/hash/index, hệ sinh thái công nghệ, và 4 kỹ thuật khai phá dữ liệu — sẽ được áp dụng trực tiếp:

- Case study: phân tích 5V trên một hệ thống dữ liệu lớn thực tế
- Bài tập: phân loại bài toán khai phá dữ liệu bằng quy trình 3 bước vừa học
- 6 mini-lab bằng Python để "chạm tay" vào từng kỹ thuật đã học

Tổng kết lý thuyết Phần 1 — bức tranh toàn cảnh

- Data Mining là gì
  - Bốn cách tiếp cận: Statistical Modeling, Machine Learning, Computational Approaches, Summarization & Feature Extraction
- Đặc trưng Big Data
  - Mô hình 5V (Volume, Velocity, Variety, Veracity, Value); vì sao cần scale-out thay vì scale-up
- Giới hạn thống kê
  - Nguyên lý Bonferroni — dữ liệu lớn dễ sinh ra "mẫu giả" nếu tìm kiếm không cẩn trọng
- Kiến thức nền
  - TF.IDF, hash function, index, secondary storage, power laws
- Bốn kỹ thuật khai phá dữ liệu
  - Classification, Clustering, Association Rules, Regression

Những điểm mấu chốt cần nhớ

Những điểm mấu chốt:

- Big Data không chỉ là "nhiều dữ liệu" — mà là dữ liệu vượt quá khả năng xử lý truyền thống theo ít nhất một trong 5V
- Tìm mẫu trong dữ liệu lớn mà không cẩn trọng thống kê (nguyên lý Bonferroni) có thể dẫn đến kết luận hoàn toàn sai lệch
- TF.IDF, hash function, index, secondary storage là công cụ nền dùng lại xuyên suốt môn học
- Bốn kỹ thuật khai phá dữ liệu nền tảng chia thành hai nhóm: có giám sát (Classification, Regression) và không giám sát (Clustering, Association Rules)

Câu hỏi ôn tập

1. Phân biệt Volume và Velocity bằng một ví dụ của riêng bạn
2. Tìm ngẫu nhiên trong 10 triệu giao dịch — nguyên lý nào giúp ước lượng số "mẫu giả" sẽ gặp?
3. Dự đoán điểm thi sinh viên nên dùng kỹ thuật nào trong 4 kỹ thuật đã học? Vì sao?
4. Vì sao Data Lake dễ trở thành "data swamp" nếu không quản lý tốt?

Tài liệu tham khảo cho Phần 1

Tài liệu	Nội dung sử dụng trong bài giảng
Han, Kamber, Pei — *Data Mining: Concepts and Techniques*	Chương 1 — khái niệm nền tảng về khai phá dữ liệu (giáo trình chính của đề cương)
Leskovec, Rajaraman, Ullman — *Mining of Massive Datasets* (mmds.org)	Chương 1 — 4 cách tiếp cận data mining, Bonferroni's Principle, TF.IDF, hash, index, secondary storage, power laws
Stanford CS246 — Mining Massive Datasets (Jure Leskovec)	Bài giảng 1 — Descriptive/Predictive methods, khung phân loại bài toán theo dữ liệu & mô hình tính toán

Vì sao những nguồn tham khảo này?

- *Mining of Massive Datasets* và slide CS246 được nhóm biên soạn **tải trực tiếp** từ các nguồn công khai chính thức (mmds.org, web.stanford.edu/class/cs246) để đảm bảo nội dung bám sát tài liệu học thuật gốc, không suy diễn
- Han–Kamber–Pei là giáo trình gốc trong đề cương chi tiết môn học đã được phê duyệt
- Cả ba nguồn đều là tài liệu **mở, được các trường đại học lớn sử dụng rộng rãi** cho các môn học tương đương môn Big Data Analytics này

THỰC HÀNH PHẦN 1

Áp dụng 5V, nguyên lý Bonferroni, và 4 kỹ thuật khai phá dữ liệu vào bài toán thực tế

